

Experiment No: 1

8-Puzzle Problem Aim

Program

```
from collections import deque
```

```
# Goal State
```

```
goal = [1, 2, 3,
```

```
        4, 5, 6,
```

```
        7, 8, 0]
```

```
# Print the puzzle as a 3x3 matrix
```

```
def print_board(state):
```

```
    for i in range(0, 9, 3):
```

```
        print(state[i], state[i+1], state[i+2])
```

```
    print()
```

```
# Generate all possible moves
```

```
def get_moves(state):
```

```
    moves = []
```

```
    pos = state.index(0)
```

```
    # Up
```

```
    if pos >= 3:
```

```
        new = state[:]
```

```
        new[pos], new[pos-3] = new[pos-3], new[pos]
```

```
        moves.append(new)
```

```
    # Down
```

```
    if pos <= 5:
```

```
        new = state[:]
```

```

        new[pos], new[pos+3] = new[pos+3], new[pos]
        moves.append(new)

# Left
if pos % 3 != 0:
    new = state[:]
    new[pos], new[pos-1] = new[pos-1], new[pos]
    moves.append(new)

# Right
if pos % 3 != 2:
    new = state[:]
    new[pos], new[pos+1] = new[pos+1], new[pos]

moves.append(new)

return moves

# BFS Algorithm
def bfs(start):
    queue = deque([start])
    visited = []

    while queue:
        state = queue.popleft()

        if state == goal:
            print("Goal State Reached!\n")
            print_board(state)
            return

        if state not in visited:

```

```
visited.append(state)
```

```
print("Current State:")
```

```
print_board(state)
```

```
for move in get_moves(state):
```

```
    if move not in visited:
```

```
        queue.append(move)
```

```
print("No Solution Found")
```

```
# User Input
```

```
print("Enter the Initial State (Use 0 for Blank):")
```

```
start = list(map(int, input().split()))
```

```
bfs(start)
```

Sample Input

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 6 7 5 8

Sample Output

Enter the Initial State (Use 0 for Blank):

1 2 3 4 0 6 7 5 8

Current State:

1 2 3

4 0 6

7 5 8

Current State:

1 0 3

4 2 6

7 5 8

Current State:

1 2 3

4 5 6

7 0 8

Current State:

1 2 3

0 4 6

7 5 8

Current State:

1 2 3

4 6 0

7 5 8

Current State:

0 1 3

4 2 6

7 5 8

Current State:

1 3 0

4 2 6

7 5 8

Current State:

1 2 3

4 5 6

0 7 8

Goal State Reached!

1 2 3

4 5 6

7 8 0

Experiment No: 2

8-Queen Problem

Program

N = 8

```
board = [[0 for _ in range(N)] for _ in range(N)]
```

```
def is_safe(row, col):
```

```
    # Check left side
```

```
    for i in range(col):
```

```
        if board[row][i] == 1:
```

```
            return False
```

```
    # Check upper diagonal
```

```
    i, j = row, col
```

```
while i >= 0 and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
        i -= 1
```

```
        j -= 1
```

```
# Check lower diagonal
```

```
i, j = row, col
```

```
while i < N and j >= 0:
```

```
    if board[i][j] == 1:
```

```
        return False
```

```
        i += 1
```

```
        j -= 1
```

```
return True
```

```
def solve(col):
```

```
    if col >= N:
```

```
        return True
```

```
for row in range(N):
```

```
    if is_safe(row, col):
```

```
        board[row][col] = 1
```

```
        if solve(col + 1):
```

```
            return True
```

```
        board[row][col] = 0
```

```
return False
```

```
if solve(0):
```

```
    print("Solution Found:\n")
```

```
    for row in board:
```

```
        print(*row)
```

```
else:
```

```
    print("No Solution Exists")
```

Sample Input

No user input required.

Sample Output

Solution Found:

```
1 0 0 0 0 0 0 0
0 0 0 0 0 0 1 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
|
```

Experiment No: 3

Water Jug Problem

Program

```
from math import gcd

def water_jug(jug1, jug2, target):
    if target > max(jug1, jug2):
        print("Target cannot be achieved.")
        return

    if target % gcd(jug1, jug2) != 0:
        print("Target cannot be achieved.")
        return
```

```
x = 0
```

```
y = 0
```

```
print("\nSteps:")
```

```
while x != target and y != target:
```

```
    if x == 0:
```

```
        x = jug1
```

```
        print(f'Fill Jug1    -> ({x}, {y})')
```

```
    elif y == jug2:
```

```
        y = 0
```

```
        print(f'Empty Jug2   -> ({x}, {y})')
```

```
    else:
```

```
        transfer = min(x, jug2 - y)
```

```
        x -= transfer
```

```
        y += transfer
```

```
        print(f'Pour Jug1 -> Jug2 -> ({x}, {y})')
```

```
print("\nTarget achieved!")
```

```
jug1 = int(input("Enter capacity of Jug1: "))
```

```
jug2 = int(input("Enter capacity of Jug2: "))
```

```
target = int(input("Enter target quantity: "))
```

```
water_jug(jug1, jug2, target)
```

Sample Input

Enter capacity of Jug1: 4

Enter capacity of Jug2: 3

Enter target quantity: 2

Sample Output

```
Enter capacity of Jug1: 4
Enter capacity of Jug2: 3
Enter target quantity: 2
```

Steps:

```
Fill Jug1      -> (4, 0)
Pour Jug1->Jug2 -> (1, 3)
Empty Jug2     -> (1, 0)
Pour Jug1->Jug2 -> (0, 1)
Fill Jug1      -> (4, 1)
Pour Jug1->Jug2 -> (2, 3)
```

Target achieved!

|

Experiment No: 4

Crypt-Arithmetic Problem Aim

Program

```
from itertools import permutations

# Read input

word1 = input("Enter First Word: ").upper()
word2 = input("Enter Second Word: ").upper()
result = input("Enter Result Word: ").upper()

letters = list(set(word1 + word2 + result))

if len(letters) > 10:
    print("Too many unique letters.")
    exit()
```

```
digits = '0123456789'
```

```
found = False
```

```
for perm in permutations(digits, len(letters)):
```

```
    table = dict(zip(letters, perm))
```

```
    # Leading letter cannot be zero
```

```
    if table[word1[0]] == '0' or table[word2[0]] == '0' or table[result[0]] == '0':
```

```
        continue
```

```
    n1 = int(''.join(table[ch] for ch in word1))
```

```
    n2 = int(''.join(table[ch] for ch in word2))
```

```
    n3 = int(''.join(table[ch] for ch in result))
```

```
    if n1 + n2 == n3:
```

```
        print("\nSolution Found\n")
```

```
        for k in sorted(table):
```

```
            print(k, "=", table[k])
```

```
    print("\n", n1)
```

```
    print("+", n2)
```

```
    print(" -----")
```

```
    print(n3)
```

```
    found = True
```

```
    break
```

```
if not found:
```

```
    print("No Solution Found.")
```

Sample Input

Enter First Word: SEND

Enter Second Word: MORE

Enter Result Word: MONEY

Sample Output

```
Enter First Word: SEND
Enter Second Word: MORE
Enter Result Word: MONEY
```

```
Solution Found
```

```
D = 7
E = 5
M = 1
N = 6
O = 0
R = 8
S = 9
Y = 2
```

```
  9567
+ 1085
-----
10652
```

Experiment No: 5

Missionaries and Cannibals Problem

Program

```
from collections import deque
```

```
# Check whether the state is valid
```

```
def is_valid(m_left, c_left):
```

```
    m_right = 3 - m_left
```

```
    c_right = 3 - c_left
```

```
    if m_left < 0 or c_left < 0 or m_left > 3 or c_left > 3:
```

```
        return False
```

```
    if (m_left > 0 and c_left > m_left):
```

```
return False
```

```
if (m_right > 0 and c_right > m_right):
```

```
    return False
```

```
return True
```

```
# BFS Algorithm
```

```
def missionaries_cannibals():
```

```
    start = (3, 3, 'L')
```

```
    goal = (0, 0, 'R')
```

```
    moves = [(1,0), (2,0), (0,1), (0,2), (1,1)]
```

```
    queue = deque([[start]])
```

```
    visited = set()
```

```
    while queue:
```

```
        path = queue.popleft()
```

```
        state = path[-1]
```

```
        if state == goal:
```

```
            print("Solution Path:\n")
```

```
            for s in path:
```

```
                print(s)
```

```
            return
```

```
        if state in visited:
```

```
            continue
```

```

visited.add(state)

m, c, boat = state

for dm, dc in moves:

    if boat == 'L':
        new_state = (m-dm, c-dc, 'R')
    else:
        new_state = (m+dm, c+dc, 'L')

    if is_valid(new_state[0], new_state[1]):
        queue.append(path + [new_state])

print("No Solution Found")

missionaries_cannibals()

```

Sample Input

No user input required.

Sample Output

Solution Path:

```

(3, 3, 'L')
(3, 1, 'R')
(3, 2, 'L')
(3, 0, 'R')
(3, 1, 'L')
(1, 1, 'R')
(2, 2, 'L')
(0, 2, 'R')
(0, 3, 'L')
(0, 1, 'R')
(1, 1, 'L')
(0, 0, 'R')

```

Experiment No: 6

VacuumCleanerProblem

Program

Vacuum Cleaner Problem using DFS

```
rooms = {}
```

```
n = int(input("Enter the number of rooms: "))
```

```
for i in range(n):
```

```
    room = input(f"Enter Room Name {i+1}: ")
```

```
    status = input(f"Enter Status of {room} (Dirty/Clean): ").capitalize()
```

```
rooms[room] = status
```

```
print("\nInitial Room Status")
```

```
print(rooms)
```

```
visited = set()
```

```
def dfs(room):
```

```
    if room in visited:
```

```
        return
```

```
    visited.add(room)
```

```
    if rooms[room] == "Dirty":
```

```
        print(f"{room} is Dirty -> Cleaning...")
```

```
        rooms[room] = "Clean"
```

```
    else:
```

```
        print(f"{room} is already Clean.")
```

```
print("\nCleaning Process")
```

```
for room in rooms:
```

```
    if room not in visited:
```

```
        dfs(room)
```

```
print("\nFinal Room Status")
```

```
print(rooms)
```

Sample Input

Enter the number of rooms: 2

Enter Room Name 1: A

Enter Status of A (Dirty/Clean): Dirty

Enter Room Name 2: B

Enter Status of B (Dirty/Clean): Dirty

Sample Output

```
Enter the number of rooms: 2
Enter Room Name 1: A
Enter Status of A (Dirty/Clean): Dirty
Enter Room Name 2: B
Enter Status of B (Dirty/Clean): Dirty
```

```
Initial Room Status
{'A': 'Dirty', 'B': 'Dirty'}
```

```
Cleaning Process
A is Dirty -> Cleaning...
B is Dirty -> Cleaning...
```

```
Final Room Status
{'A': 'Clean', 'B': 'Clean'}
|
```

Experiment No: 7

Breadth First Search (BFS)

Program

```
from collections import deque

# Breadth First Search Function
def bfs(graph, start, goal):
```

```
queue = deque([(start, [start])])
```

```
visited = set()
```

```
while queue:
```

```
    node, path = queue.popleft()
```

```
    if node == goal:
```

```
        return path
```

```
    if node in visited:
```

```
        continue
```

```
    visited.add(node)
```

```
    for neighbour in graph[node]:
```

```
        if neighbour not in visited:
```

```
            queue.append((neighbour, path + [neighbour]))
```

```
return None
```

```
# Main Program
```

```
graph = {
```

```
    0: [1, 2],
```

```
    1: [0, 2, 3],
```

```
    2: [0, 1, 3],
```

```
    3: [1, 2, 4],
```

```

    4: [3]
}

print("Graph Representation")
print("-----")

for node in graph:
    print(node, "->", graph[node])

print()

start = int(input("Enter Source Node : "))
goal = int(input("Enter Destination Node : "))

path = bfs(graph, start, goal)

print()

if path:

    print("Breadth First Search Successful")
    print("-----")
    print("Source Node :", start)
    print("Destination Node :", goal)
    print("Shortest Path :", " -> ".join(map(str, path)))
    print("Number of Nodes Visited :", len(path))

else:
    print("No Path Found")

```

Sample Input

Enter Source Node 0

Enter Destination Node : 4

Sample Output

```
-----  
Graph Representation  
-----
```

```
0 -> [1, 2]  
1 -> [0, 2, 3]  
2 -> [0, 1, 3]  
3 -> [1, 2, 4]  
4 -> [3]
```

```
Enter Source Node      : 0  
Enter Destination Node : 4
```

```
Breadth First Search Successful  
-----
```

```
Source Node      : 0  
Destination Node : 4  
Shortest Path    : 0 -> 1 -> 3 -> 4  
Number of Nodes Visited : 4  
|
```

Experiment No: 8

Depth First Search (DFS)

Program

```
# Depth First Search (DFS)
```

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F'],  
    'D': [],
```

```

'E': ['F'],
'F': []
}

visited = set()

def dfs(node):

    if node not in visited:

        print(node, end=" ")

        visited.add(node)

        for neighbour in graph[node]:
            dfs(neighbour)

print("Graph:")
for node in graph:
    print(node, "->", graph[node])

print("\n\nDFS Traversal:")

dfs('A')

```

Sample Input

Starting Node : A

(The graph is predefined in the program.)

Sample Output

```
Graph:
A -> ['B', 'C']
B -> ['D', 'E']
C -> ['F']
D -> []
E -> ['F']
F -> []
```

```
DFS Traversal:
A B D E F C
```

Experiment No: 9

Travelling Salesman Problem (TSP)

Program

```
import numpy as np
```

```
def tsp(cities):
```

```

n = len(cities)

# Create Distance Matrix
distance = np.zeros((n, n))

for i in range(n):
    for j in range(n):
        if i != j:
            distance[i][j] = np.linalg.norm(cities[i] - cities[j])

visited = [False] * n
visited[0] = True

tour = [0]
total_distance = 0
current = 0

while len(tour) < n:

    nearest = -1
    minimum = float('inf')

    for city in range(n):

        if not visited[city] and distance[current][city] < minimum:
            minimum = distance[current][city]
            nearest = city

    tour.append(nearest)

```

```
visited[nearest] = True  
total_distance += minimum  
current = nearest
```

```
total_distance += distance[current][0]  
tour.append(0)
```

```
return tour, total_distance
```

```
# Main Program
```

```
n = int(input("Enter Number of Cities: "))
```

```
cities = []
```

```
for i in range(n):  
    x = float(input(f"Enter x-coordinate of City {i}: "))  
    y = float(input(f"Enter y-coordinate of City {i}: "))  
    cities.append(np.array([x, y]))
```

```
tour, distance = tsp(cities)
```

```
print("\nOptimal Tour")
```

```
print(" -> ".join(map(str, tour)))
```

```
print("Total Distance =", round(distance,2))
```

Sample Input

Enter Number of Cities: 4

City 0 : (0,0)

City 1 : (2,3)

City 2 : (5,2)

City 3 : (6,6)

Sample Output

```
Enter Number of Cities: 4
Enter x-coordinate of City 0: 0
Enter y-coordinate of City 0: 0
Enter x-coordinate of City 1: 2
Enter y-coordinate of City 1: 3
Enter x-coordinate of City 2: 5
Enter y-coordinate of City 2: 2
Enter x-coordinate of City 3: 6
Enter y-coordinate of City 3: 6
```

```
Optimal Tour
0 -> 1 -> 2 -> 3 -> 0
Total Distance = 19.38
```

EXPERIMENT 10:

A* algorithm.

Program:

```
import heapq

def a_star(graph, heuristic, start, goal):
    queue = [(heuristic[start], 0, start, [start])]
    visited = set()

    while queue:
        f, cost, node, path = heapq.heappop(queue)

        if node == goal:
            return path, cost

        if node in visited:
            continue

        visited.add(node)

        for neighbour, distance in graph[node]:
            if neighbour not in visited:
                new_cost = cost + distance
                f_value = new_cost + heuristic[neighbour]

                heapq.heappush(
                    queue,
                    (f_value, new_cost, neighbour,
                     path + [neighbour])
                )

    return None, None

# Graph with edge costs
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('D', 2), ('E', 5)],
    'C': [('E', 1)],
    'D': [('F', 3)],
    'E': [('F', 2)],
    'F': []
}
```

```
# Heuristic values
heuristic = {
    'A': 5,
    'B': 4,
    'C': 3,
    'D': 3,
    'E': 2,
    'F': 0
}

start = 'A'
goal = 'F'

path, cost = a_star(graph, heuristic, start, goal)

if path:
    print("Shortest Path:", " -> ".join(path))
    print("Total Cost:", cost)
else:
    print("Path not found")
```

Sample Output:

```
Shortest Path: A -> B -> D -> F
Total Cost: 6
```

Experiment 11:

Map Coloring Using CSP:

Program:

```
# Map Coloring using CSP
```

```
regions = ['A', 'B', 'C', 'D']
```

```
# Adjacent regions
```

```
neighbors = {  
    'A': ['B', 'C'],  
    'B': ['A', 'C', 'D'],  
    'C': ['A', 'B', 'D'],  
    'D': ['B', 'C']  
}
```

```
# Available colors
```

```
colors = ['Red', 'Green', 'Blue']
```

```
assignment = {}
```

```
def is_safe(region, color):
```

```
    for neighbor in neighbors[region]:  
        if neighbor in assignment:  
            if assignment[neighbor] == color:  
                return False  
    return True
```

```
def map_coloring(index):
```

```
    # All regions are colored  
    if index == len(regions):  
        return True
```

```
    region = regions[index]
```

```
    for color in colors:
```

```
        if is_safe(region, color):  
            assignment[region] = color
```

```
            if map_coloring(index + 1):  
                return True
```

```
            # Backtracking
```

```
            del assignment[region]
```

```
    return False
```

```
# Solve CSP
```

```
if map_coloring(0):
```

```
print("Map Coloring Solution:")

for region in regions:
    print("Region", region, ":", assignment[region])

else:
    print("No solution found")
```

Sample Output:

```
Map Coloring Solution:
Region A : Red
Region B : Green
Region C : Blue
Region D : Red
```

Experiment 12:

Tic Tac Toe game

Tic-Tac-Toe Game

```
board = [' ' for i in range(9)]
```

```
def display_board():
```

```
    print()
    print(board[0], "|", board[1], "|", board[2])
    print("--+---+--")
    print(board[3], "|", board[4], "|", board[5])
    print("--+---+--")
    print(board[6], "|", board[7], "|", board[8])
    print()
```

```
def check_winner(player):
```

```
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
```

```
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
```

```
    return False
```

```
def play_game():
```

```
    player = 'X'
```

```
    for turn in range(9):
        display_board()
```

```
        print("Player", player)
```

```
        try:
```

```
            position = int(input("Enter position (1-9): ")) - 1
        except ValueError:
            print("Please enter a number from 1 to 9.")
            continue
```

```

if position < 0 or position > 8:
    print("Invalid position!")
    continue

if board[position] != ' ':
    print("Position already occupied!")
    continue

board[position] = player

if check_winner(player):
    display_board()
    print("Player", player, "wins!")
    return

if player == 'X':
    player = 'O'
else:
    player = 'X'

display_board()
print("Game Draw!")

```

play_game()

Sample Output:

```

| | |
+---+
| | |
+---+
| | |

```

Player X
Enter position (1-9): 1

```

X | |
+---+
| | |
+---+
| | |

```

Player O
Enter position (1-9): 5